

# JSX, composants fonctionnels, props et state

## Objectifs de cette section

Dans cette section, vous allez apprendre à construire vos **premiers composants React**. Vous découvrirez la syntaxe JSX, les props, et le hook `useState` pour gérer l'état local d'un composant. À la fin de cette section, vous serez capable de :

- Créer un composant fonctionnel React avec JSX.
- Utiliser les **props** pour transmettre des données.
- Gérer un **state local** avec `useState`.
- Mettre à jour dynamiquement l'interface selon l'état.

## 1. JSX : une syntaxe déclarative proche du HTML

JSX permet de décrire une interface en React de manière lisible et proche du HTML. Mais il y a quelques différences :

- Un seul élément parent doit être retourné.
- On utilise `className` au lieu de `class`.
- Les expressions JS s'écrivent entre `{}`.

### Exemple : composant avec JSX

```
function Hello() {
  return (
    <div>
      <h1>Bonjour React</h1>
      <p>Ceci est un composant JSX.</p>
    </div>
  );
}
```

## 2. Composants fonctionnels

Un composant React est une **fonction** qui retourne du JSX. Il peut :

- Recevoir des **props** (données externes).
- Gérer un **state** (données internes, variables d'état).
- Avoir une logique métier ou des effets secondaires (plus tard).

### Exemple : composant avec props

```
function Greeting({name}) {
  return <p>Bonjour {name} !</p>;
}

// Utilisation
<Greeting name="Alice"/>
```

## 3. Gérer l'état avec `useState`

Le **state** permet de faire varier dynamiquement l'interface. `useState` est un **hook** qui retourne :

- Une variable d'état.
- Une fonction pour la mettre à jour.

### Exemple : compteur React

```
import {useState} from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Compteur : {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

## 4. Affichage conditionnel et modification de state

Avec React, on peut afficher un contenu ou un autre selon l'état, très simplement :

- Grâce à des expressions ternaires dans le JSX.
- Ou en encapsulant le rendu dans une fonction.

### Exemple : bouton toggle affichage

```
import {useState} from "react";

function ToggleText() {
  const [visible, setVisible] = useState(true);

  return (
    <div>
      {visible && <p>Ce texte peut être masqué.</p>}
      <button onClick={() => setVisible(!visible)}>
        {visible ? "Masquer" : "Afficher"}
      </button>
    </div>
  );
}
```

## 5. Exercice de mise en pratique

### Consignes

1. Créez un composant `TaskItem` qui :
  - Reçoit une prop `text`.
  - Affiche cette tâche dans un paragraphe.
  - Contient un bouton "Masquer".
2. Lorsqu'on clique sur le bouton, la tâche **disparaît** grâce à un `useState`.
3. Créez un composant `App` qui affiche **trois** `TaskItem` avec des textes différents.

### Organisation attendue

```
/exercices
├── 11-react-taskitem-props-state
│   ├── App.jsx
│   ├── TaskItem.jsx
│   └── main.jsx
```